



Lab3
Machine Learning on Spark

Nhóm

Thành viên:

Phan Bá Đức	22120071
Đặng Duy Lâm	22120182
Nguyễn Quốc Thịnh	22120347
Trần Đức Thịnh	22120348

Giáo viên hướng dẫn:

- Nguyễn Ngọc Thảo
- Huỳnh Hải Lâm Đăng
- Lê Ngọc Thành

Mục Lục

Phân chia công việc nhóm	4
I. Classification with Logistic Regression	5
A. Structured API Implementation (High-Level)	5
1. Giới thiệu.....	5
2. Các hàm chính	5
3. Các bước xử lý chính.....	6
B. MLLib RDD-Based Implementation	7
1. Các hàm chính	7
2. Các bước xử lý chính.....	8
C. Low-Level Operations	9
1. Chuẩn bị và Tiền xử lý Dữ liệu	9
2. Xây dựng Mô hình Hồi quy Logistic với SGD.....	9
3. Huấn luyện và Đánh giá Mô hình.....	10
4. Huấn luyện và Đánh giá:	10
5. Phân tích Kết quả	11
6. So sánh với MLLib RDD-Based	11
II. Regression with Decision Trees.....	12
A. Structured API Implementation (High-Level)	12
1. Loại bỏ các dòng chứa giá trị thiếu (null):	12
2. Lọc ra các đặc trưng số học (numeric features):.....	12
3. Chuyển đổi các cột đặc trưng thành vector đầu vào:	12
4. Tách dữ liệu thành tập huấn luyện và kiểm tra:.....	12
B. MLLib RDD-Based Implementation	13
1. Đọc dữ liệu từ HDFS:	13
2. Xác định các cột số	13
3. Lọc bỏ dòng tiêu đề:	13
4. Chuyển đổi kiểu dữ liệu:.....	14
5. Tạo RDD LabeledPoint:	14
6. Chia dữ liệu thành train/test:	14
C. Low-Level Operations	14
1. Ý tưởng thực hiện thuật toán hồi quy bằng cây quyết định.....	14
2. Các bước của thuật toán	15
3. Triển khai code	17
Trực quan hình ảnh cây quyết định: Xem tại file <i>CorrelationMatrix.ipynb</i>	21

Thách thức khi mở rộng phương pháp Decision Tree với Variance Reduction cho cây sâu hơn và rừng cây (ensemble)	22
---	-----------

Phân chia công việc nhóm

MSSV	Họ và Tên	Công việc
22120071	Phan Bá Đức	- Thực hiện Low_Level (Regression) - Viết file readme và báo cáo về phần bài làm của mình - Tổng hợp file báo cáo của các thành viên và viết file readme
22120182	Đặng Duy Lâm	- Thực hiện Structured_API và MLlib_RDD_Based (Regression) - Viết báo cáo về phần bài làm của mình
22120347	Nguyễn Quốc Thịnh	- Thực hiện Low_Level (Classification) - Viết báo cáo về phần bài làm của mình
22120348	Trần Đức Thịnh	- Thực hiện Structured_API và MLlib_RDD_Based (Classification) - Viết báo cáo về phần bài làm của mình

I. Classification with Logistic Regression

Dataset: [Credit Card Fraud Detection](#)

A. Structured API Implementation (High-Level)

1. Giới thiệu

Phân tích và xây dựng mô hình học máy phân loại nhị phân trên bộ dữ liệu creditcard.csv bằng cách sử dụng Structured API của Apache Spark với Python (PySpark). File Structured_API.ipynb chứa mã nguồn để đọc dữ liệu từ HDFS, tiền xử lý dữ liệu, chuẩn hóa, huấn luyện mô hình hồi quy logistic, và đánh giá hiệu suất. Báo cáo này trình bày các hàm chính được sử dụng trong đoạn mã, mục đích của chúng, và các bước xử lý dữ liệu chính.

2. Các hàm chính

Hàm check_label_ratio

Hàm check_label_ratio được định nghĩa để tính toán số lượng và tỉ lệ các nhãn trong một PySpark DataFrame.

def check_label_ratio(df):

label_counts_agg_df = df.groupBy("Class").count()

collected_counts = label_counts_agg_df.collect()

result = []

for row in collected_counts:

label = row["Class"]

count_val = row["count"]

*ratio = (count_val / df.count() * 100)*

result.append({

"label": label,

"count": count_val,

"ratio": f"{ratio:.3f}%"

})

result.sort(key=lambda x: x['label'])

return result

Mục đích:

- Phân tích phân bố nhãn: Xác định số lượng và tỉ lệ các nhãn (lớp 0 và 1) trong tập dữ liệu để đánh giá mức độ mất cân bằng, điều này quan trọng trong bài toán phân loại nhị phân như phát hiện gian lận.
- Hỗ trợ đánh giá dữ liệu: Kết quả giúp quyết định xem có cần áp dụng kỹ thuật xử lý mất cân bằng (như oversampling hoặc undersampling) hay không.
- Lưu trữ thông tin: Tỉ lệ nhãn được lưu vào file JSON để báo cáo và phân tích sau này.

Cách hoạt động:

- Nhóm dữ liệu theo cột Class và đếm số lượng mẫu cho mỗi nhãn bằng groupBy và count.
- Thu thập kết quả dưới dạng danh sách các hàng.
- Tính tỉ lệ phần trăm của mỗi nhãn so với tổng số mẫu.
- Sắp xếp kết quả theo nhãn và trả về danh sách các dictionary chứa thông tin nhãn, số lượng, và tỉ lệ.

3. Các bước xử lý chính

Ngoài hàm check_label_ratio, đoạn mã thực hiện một chuỗi các thao tác để đạt được mục tiêu phân loại. Dưới đây là các bước chính và mục đích của chúng:

a) Khởi động HDFS và YARN

Đoạn mã bắt đầu bằng việc khởi động các dịch vụ Hadoop Distributed File System (HDFS) và YARN bằng lệnh start-dfs.sh và start-yarn.sh.

Mục đích: Thiết lập môi trường phân tán để lưu trữ và xử lý dữ liệu lớn, cho phép đọc dữ liệu từ HDFS và tận dụng tài nguyên tính toán của YARN.

b) Khởi tạo SparkSession

Tạo SparkSession với tên ứng dụng Structured_API để quản lý các tác vụ Spark.

Mục đích: Cung cấp điểm truy cập chính để sử dụng Structured API của Spark, cho phép thao tác trên DataFrame và thực hiện các tác vụ học máy.

c) Đọc dữ liệu từ HDFS

Đọc file creditcard.csv từ HDFS

(hdfs://localhost:9000/hcmus/22120348/creditcard.csv)

vào DataFrame với tùy chọn header=True và inferSchema=True.

Mục đích: Tải dữ liệu vào môi trường Spark để xử lý, với schema được tự động suy ra để Ascending Order để đảm bảo dữ liệu được sắp xếp đúng.

d) Tiền xử lý dữ liệu

(1) Loại bỏ cột không cần thiết

Loại bỏ các cột Time và Amount khỏi DataFrame.

Mục đích: Chỉ giữ lại các cột chứa các thành phần chính (principal components) và nhãn Class, loại bỏ các đặc trưng không liên quan để giảm kích thước dữ liệu và tập trung vào các đặc trưng quan trọng.

(2) Kiểm tra dữ liệu rỗng

Kiểm tra sự tồn tại của giá trị rỗng hoặc NaN trong DataFrame bằng cách sử dụng count và when.

Mục đích: Đảm bảo dữ liệu không chứa giá trị rỗng, giúp tránh lỗi trong các bước xử lý tiếp theo.

(3) Kiểm tra và loại bỏ dữ liệu trùng lặp

Tính toán số lượng bản ghi trùng lặp và loại bỏ chúng bằng phương thức distinct().

Mục đích: Loại bỏ các bản ghi trùng lặp để đảm bảo chất lượng dữ liệu, tránh thiên lệch trong mô hình, và giảm kích thước dữ liệu.

(4) Tạo vector đặc trưng

Sử dụng `VectorAssembler` để kết hợp các cột đặc trưng thành một cột vector features.

Mục đích: Chuẩn bị dữ liệu đầu vào cho mô hình học máy bằng cách chuyển các đặc trưng thành định dạng vector, phù hợp với yêu cầu của thư viện MLlib.

e) Chia tập dữ liệu

Chia `DataFrame` thành tập huấn luyện (80%) và tập kiểm tra (20%) với `randomSplit` và seed cố định.

Mục đích: Tạo tập huấn luyện để huấn luyện mô hình và tập kiểm tra để đánh giá hiệu suất, với seed đảm bảo tính tái lập.

f) Chuẩn hóa dữ liệu

Sử dụng `StandardScaler` để chuẩn hóa cột *features* thành *scaled_features* với trung bình bằng 0 và độ lệch chuẩn bằng 1.

Mục đích: Đưa các đặc trưng về cùng thang đo để cải thiện hiệu suất và tốc độ hội tụ của mô hình hồi quy logistic.

g) Huấn luyện mô hình hồi quy logistic

Huấn luyện mô hình `LogisticRegression` với nhiều bộ tham số (*maxIter* và *learning rate*), sử dụng cột *scaled_features* và *Class*.

Mục đích: Xây dựng mô hình phân loại nhị phân để dự đoán nhãn *Class*, tối ưu hóa hiệu suất bằng cách thử nghiệm các tham số khác nhau.

h) Đánh giá mô hình

Đánh giá mô hình bằng các chỉ số `areaUnderROC` (AUC), accuracy, F1-score, precision, recall, và tính toán ma trận nhầm lẫn (TP, TN, FP, FN).

Mục đích: Đánh giá khả năng dự đoán của mô hình trên tập kiểm tra, so sánh hiệu suất giữa các bộ tham số để chọn mô hình tốt nhất.

i) Lưu kết quả

Lưu thông tin phân bố nhãn (train và test) và kết quả đánh giá mô hình vào file JSON (`Structured_API.json`).

Mục đích: Lưu trữ kết quả phân tích và hiệu suất mô hình để báo cáo, phân tích sau này, hoặc sử dụng trong các ứng dụng khác.

j) Dừng dịch vụ

Dừng `SparkSession`, HDFS, và YARN bằng `spark.stop()` và lệnh `stop-dfs.sh`, `stop-yarn.sh`.

B. MLlib RDD-Based Implementation

1. Các hàm chính

Hàm `row_to_labeled_point`

Hàm `row_to_labeled_point` chuyển đổi một hàng dữ liệu (dưới dạng tuple)

thành đối tượng LabeledPoint, một cấu trúc dữ liệu cần thiết cho thư viện MLlib của Spark.

Mục đích:

- Chuẩn bị dữ liệu đầu vào cho các thuật toán học máy của MLlib, đảm bảo định dạng phù hợp.
- Tách nhãn (cột Class) và các đặc trưng, giúp mô hình dễ dàng nhận diện mẫu dữ liệu.
- Chuyển đổi kiểu dữ liệu sang float cho đặc trưng và int cho nhãn để tránh lỗi khi huấn luyện.

Cách hoạt động: Hàm nhận một tuple dữ liệu, trích xuất nhãn từ cột Class, tạo vector đặc trưng từ các cột còn lại, và trả về đối tượng LabeledPoint.

Hàm check_label_ratio (như trong phần Structured API Implementation)

2. Các bước xử lý chính

Ngoài hai hàm trên, đoạn mã thực hiện nhiều thao tác quan trọng để đạt được mục tiêu của tôi. Dưới đây là các bước chính:

a) Cài đặt môi trường

Đoạn mã cài đặt Java, Apache Spark (phiên bản 3.5.5), thiết lập biến môi trường, và cài đặt thư viện findspark.

b) Khởi tạo SparkSession và SparkContext

Tạo SparkSession với tên ứng dụng LowLevelDecisionTreeNYCTaxi và SparkContext để làm việc với RDD.

c) Đọc và xử lý dữ liệu

Đọc file creditcard.csv vào RDD, tách tiêu đề, chuyển đổi dữ liệu thành danh sách giá trị float, và loại bỏ cột không cần thiết (cột 0 và 29).

Mục đích: Dữ liệu thô được làm sạch và định dạng đúng để chuẩn bị cho các bước tiếp theo như chuẩn hóa và huấn luyện.

d) Kiểm tra và loại bỏ dữ liệu trùng lặp

Kiểm tra số lượng bản ghi trùng lặp và loại bỏ chúng bằng phương thức distinct().

Mục đích: Loại bỏ trùng lặp để đảm bảo chất lượng dữ liệu, tránh thiên lệch trong mô hình, và giảm kích thước dữ liệu.

e) Chuẩn hóa dữ liệu

Sử dụng StandardScaler để chuẩn hóa đặc trưng về z-score (trừ trung bình, chia độ lệch chuẩn).

Mục đích: Các đặc trưng có cùng thang đo để cải thiện hiệu suất và tốc độ hội tụ của mô hình hồi quy logistic.

f) Chia tập dữ liệu

Chia dữ liệu thành tập huấn luyện (80%) và kiểm tra (20%) với seed cố định.

g) Huấn luyện mô hình hồi quy logistic

Huấn luyện mô hình LogisticRegressionWithSGD với nhiều bộ tham số (learning rate và số vòng lặp), đánh giá bằng AUC, accuracy, recall, và ma trận nhầm lẫn.

h) Lưu kết quả

Lưu phân bố nhãn và kết quả đánh giá mô hình vào file JSON.

C. Low-Level Operations

1. Chuẩn bị và Tiền xử lý Dữ liệu

a) Tải và Giải nén Dữ liệu

Tải xuống file creditcard.zip từ Kaggle và giải nén nó ở cùng thư mục với file notebook.

b) Khởi tạo Spark

- findspark.init(): sử dụng để đảm bảo PySpark có thể được import và sử dụng đúng cách trong môi trường.
- Một SparkSession được khởi tạo với tên ứng dụng "ClassificationRDD" và được sử dụng cho các thao tác RDD.
- Cấu hình logging được thiết lập để giảm bớt thông báo, chỉ hiển thị lỗi (log4j.setLevel(log4j.Level.ERROR))

c) Đọc và Định dạng Dữ liệu (Bước 4-5)

- Đọc file creditcard.csv được bằng sc.textFile(), tạo ra một RDD trong đó mỗi phần tử là một dòng của file CSV.
 - Dòng tiêu đề (header) được trích xuất.
 - Dữ liệu được xử lý bằng các phép biến đổi RDD:
 - filter(): Loại bỏ dòng tiêu đề khỏi RDD dữ liệu.
 - map(): Phân tích mỗi dòng.
 - line.split(","): Tách dòng thành các chuỗi giá trị.
 - Tạo rdd_data"
 - Các giá trị đặc trưng (features) được chuyển đổi thành kiểu float.
 - Nhãn được chuyển đổi thành kiểu int (0, 1), và được loại bỏ cặp dấu ngoặc kép.
 - Chuyển đổi mỗi dòng thành một tuple (features_list, label) sử dụng map().
- **Kết quả:** rdd_data là một RDD chứa các cặp (danh sách các đặc trưng, nhãn).

2. Xây dựng Mô hình Hồi quy Logistic với SGD

Triển khai một mô hình Hồi quy Logistic sử dụng thuật toán Tối ưu Hóa Độ Dốc Ngẫu Nhiên (Stochastic Gradient Descent - SGD) thông qua lớp ClassifierSGD.

- __init__: Khởi tạo các tham số mô hình như learning_rate, n_epoch (số vòng lặp), sz_batch (kích thước batch), freq_log. Trọng số w và độ chệch bias được khởi tạo ngẫu nhiên.

- `_sigmoid`: Hàm sigmoid, dùng để ép giá trị đầu ra của mô hình vào khoảng (0, 1).
- `_dot_product`: Tính tích vô hướng giữa hai vector.
- `_predict` (cho một mẫu): Tính toán đầu ra dự đoán $z = w \cdot x + b$ và áp dụng hàm sigmoid.
- `_compute_gradients`: Tính toán gradient của hàm mất mát (log loss) đối với trọng số w và bias cho một batch dữ liệu.
- `_update_weights`: Cập nhật trọng số w và bias dựa trên gradient và learning rate.
- `_create_batches`: Chia RDD đầu vào thành các batch nhỏ hơn.
- `fit(data_rdd)`: Hàm huấn luyện chính, có tổng cộng `n_epoch` epoch. Trong mỗi epoch:
 - Xáo trộn dữ liệu.
 - Tạo các batch từ dữ liệu đã xáo trộn.
 - Tính toán gradient cho từng batch.
 - Tổng hợp và tính trung bình các gradient từ tất cả các batch.
 - Cập nhật trọng số và bias của mô hình.
- `predict(features_rdd)`: Áp dụng mô hình đã huấn luyện lên một RDD các đặc trưng để đưa ra dự đoán (0 hoặc 1 dựa trên ngưỡng 0.5).

3. Huấn luyện và Đánh giá Mô hình.

Xây dựng lớp `MetricEvaluator` để tính toán các chỉ số đánh giá hiệu suất mô hình từ RDD dự đoán và RDD nhãn thực tế.

- `_get_confusion_matrix`: Tính toán True Positives (TP), False Positives (FP), False Negatives (FN), True Negatives (TN) dựa trên một ngưỡng (mặc định là 0.5).
- Các thông số đánh giá:
 - Accuracy
 - Precision
 - Recall
 - Specificity
 - F1-Score.
- `print_metrics`: In các chỉ số đánh giá ra màn hình.

4. Huấn luyện và Đánh giá:

- `rdd_data` được chia thành tập huấn luyện (`train_rdd`) và tập kiểm tra (`test_rdd`) với tỷ lệ 80/20.
- Tạo mô hình `sgd_clf` từ lớp `ClassifierSGD` được khởi tạo với các siêu tham số:
 - `learning_rate=0.001`
 - `n_epoch=20`
 - `sz_batch=64`
- Gọi `sgd_clf.fit(train_rdd)` để huấn luyện mô hình.

- Mô hình được áp dụng lên tập kiểm tra để tạo ra `predictions_rdd`.
- Sử dụng `MetricEvaluator` để tính toán và in ra các thông số đánh giá trên tập kiểm tra.
- Kết quả đánh giá mô hình trên tập kiểm tra cho thấy:
 - Accuracy: 0.9982
 - Precision: 0.0000
 - Recall: 0.0000
 - F1 Score: 0.0000
 - Specificity: 1.0000

5. Phân tích Kết quả

- **Accuracy cao không đảm bảo mô hình hiệu quả:** Trong bộ dữ liệu bất cân bằng nghiêm trọng (tỷ lệ giao dịch gian lận chỉ khoảng 0,17 %), mô hình luôn dự đoán “không gian lận” vẫn đạt độ chính xác xấp xỉ 99,8 %. Do đó, chỉ dựa vào accuracy dễ dẫn tới đánh giá sai về năng lực phân biệt của mô hình.
- **Precision, Recall, F1 bằng 0 – không phát hiện được giao dịch gian lận?** Việc precision và recall bằng 0 cho thấy mô hình không tạo ra bất kỳ dự đoán “gian lận” nào (True Positive = 0). Điều này ngụ ý mô hình chưa “học” được đặc trưng của lớp thiểu số và hoàn toàn bỏ qua mục tiêu chính: phát hiện gian lận.
- **Specificity cao nhưng ít ý nghĩa khi mục tiêu là phát hiện gian lận:** Specificity bằng 1,0 chỉ khẳng định mọi giao dịch “không gian lận” đều được phân loại chính xác, nhưng lại không hỗ trợ gì cho việc phát hiện các trường hợp gian lận.

6. So sánh với MLlib RDD-Based

Với $LR = 0.1$, $Iteration = 50$ thì:

Metric	Structured API	MLlib RDD-Based	Low-level Operations
Accuracy	0.9984	0.7543	0.9982
Precision	0.9020	0.0068	0.0000
Recall	0.5412	0.9485	0.0000
F1-Score	0.6765	0.0135	0.0000
Specificity	0.9999	0.7538	1.0000

- **Structured API:** Độ chính xác rất cao (Accuracy - 99.84%, Precision = 90.20%), tuy nhiên khả năng bỏ qua gian lận cao
- **MLlib RDD-Based:** Khả năng bắt gian lận thế cao (với Recall = 94.85%), tuy nhiên với độ chính xác tổng thể thấp hơn (Accuracy = 75.43%), dẫn đến nhiều trường hợp dương tính giả hơn.
- **Low-level Operations:** Độ chính xác cao (Accuracy = 99.82%), tuy nhiên lại không có ích trong việc xác định gian lận do không dự đoán giá trị lừa đảo (Precision = 0, Recall = 0)

→ Việc áp dụng thuật toán phân loại cơ bản đối với bộ dữ liệu có tính đặc thù cao (tỉ lệ giữa lừa đảo và không lừa đảo rất chênh lệch) không hiệu quả. Thư viện MLib cho phép thay đổi tham số và cung cấp nhiều công cụ để xử lý bộ dữ liệu để cho ra mô hình tốt hơn.

II. Regression with Decision Trees

Dataset: [New York City Taxi Trip Duration](#)

A. Structured API Implementation (High-Level)

(Dữ liệu được được từ hdfs triển khai bằng Docker)

Hàm preprocess() thực hiện tiền xử lý dữ liệu để chuẩn bị cho việc huấn luyện mô hình học máy. Cụ thể, các bước xử lý được thực hiện như sau:

1. Loại bỏ các dòng chứa giá trị thiếu (null):

Hàm sử dụng dropna để loại bỏ các dòng mà trong đó có ít nhất một giá trị thiếu ở các cột đặc trưng (features) hoặc cột nhãn (label). Đây là bước quan trọng để đảm bảo rằng mô hình học máy không bị lỗi khi gặp dữ liệu không đầy đủ.

2. Lọc ra các đặc trưng số học (numeric features):

Duyệt qua toàn bộ các cột trong schema của DataFrame và giữ lại những cột có kiểu dữ liệu số. Sau đó, loại bỏ cột nhãn (trip_duration) ra khỏi danh sách này, để đảm bảo rằng chỉ có các đặc trưng đầu vào được đưa vào mô hình.

3. Chuyển đổi các cột đặc trưng thành vector đầu vào:

Sử dụng VectorAssembler để gộp các cột đặc trưng số học lại thành một cột vector duy nhất có tên là "features".

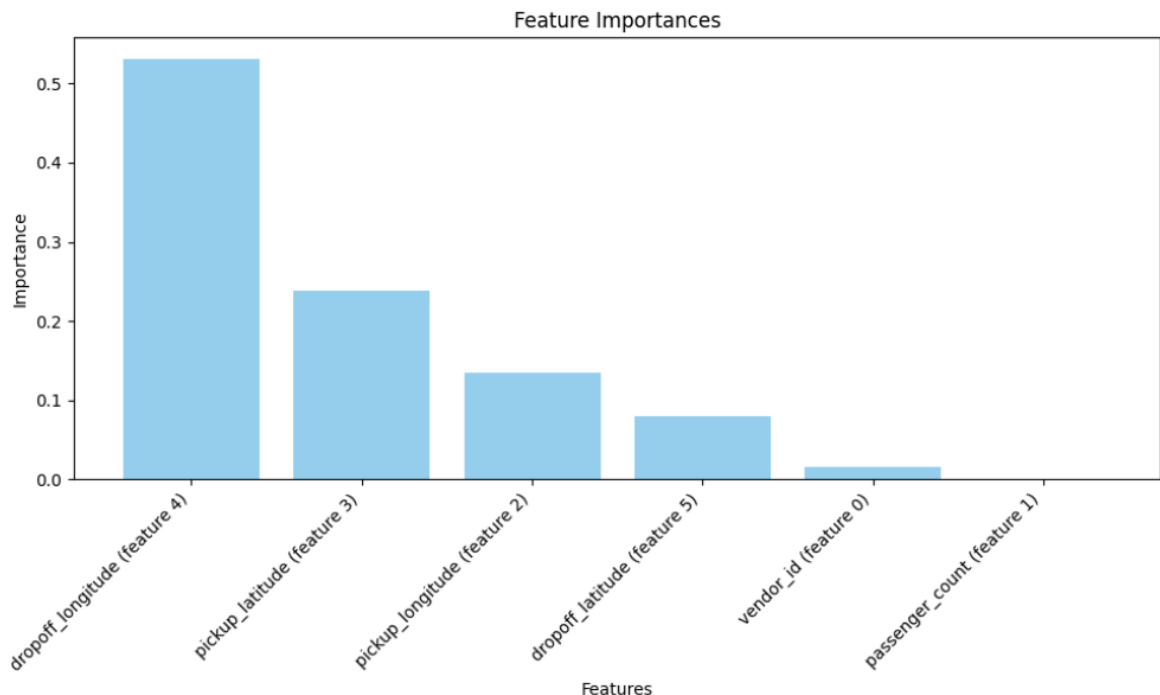
4. Tách dữ liệu thành tập huấn luyện và kiểm tra:

Dữ liệu sau khi xử lý được chia thành hai phần: 80% cho huấn luyện và 20% cho kiểm tra.

Sau đó, em thử train model với độ sâu cây: 3, 5, 7 thì kết quả cho ra (RMSE và R2) khá tương đồng nhau:

Max depth	RMSE	R2
3	6288.530004971957	0.004556815634278011
5	6288.359505585617	0.004610793320991791
7	6333.8885416933335	-0.0098550370776469

Dựa trên các plot của feature importances, 4 đặc trưng luôn có ảnh hưởng mạnh nhất tới cây đó chính là 4 đặc trưng về vị trí địa lý (pickup_longitudes, pickup_latitudes, dropoff_longitudes, dropoff_latitudes)



Hình: plot feature importances của cây $max_depth=7$

Có thể thấy điều này khá hợp lý, tuy nhiên mô hình vẫn cho ra kết quả RMSE và R2 tương đối thấp, điều này có thể là do mô hình decision tree dễ bị overfit trên dữ liệu huấn luyện. Điều này có thể khiến mô hình học quá kỹ các đặc điểm cụ thể của tập huấn luyện thay vì khái quát hóa cho dữ liệu chưa thấy. Ngoài ra, một số nguyên nhân khác có thể là do tiền xử lý quá đơn giản, chưa đủ tốt,....

B. MLlib RDD-Based Implementation

(Dữ liệu được đọc từ hdfs triển khai bằng Docker)

Ở mục này, bước tiền xử lý sẽ thực hiện những công việc sau:

1. Đọc dữ liệu từ HDFS:

Dữ liệu được đọc từ tệp train.csv lưu trữ trên HDFS thông qua `sc.textFile`. Dòng đầu tiên được lấy ra làm header để xác định các cột.

2. Xác định các cột số

Chỉ số của các cột dạng số được chọn thủ công do đã biết từ cách trước (Structured API), bao gồm: `vendor_id`, `passenger_count`, `pickup_longitude`, `pickup_latitude`, `dropoff_longitude`, `dropoff_latitude`, và `trip_duration` (nhân mục tiêu).

3. Loại bỏ dòng tiêu đề:

Dòng tiêu đề được loại bỏ khỏi tập dữ liệu để tránh gây lỗi khi chuyển đổi kiểu dữ liệu.

4. Chuyển đổi kiểu dữ liệu:

Mỗi dòng còn lại được tách và chuyển từng cột đã chọn thành kiểu float, đảm bảo rằng dữ liệu phù hợp với yêu cầu của thư viện MLlib.

5. Tạo RDD LabeledPoint:

Mỗi dòng sau khi chuyển đổi được ánh xạ thành một đối tượng LabeledPoint, trong đó:

- Cột trip_duration là **label** (biến mục tiêu).
- Các cột còn lại là **features** (biến đầu vào dưới dạng vector).

6. Chia dữ liệu thành train/test:

Dữ liệu RDD được chia ngẫu nhiên thành hai phần:

- 80% cho tập huấn luyện (train_data).
- 20% cho tập kiểm tra (test_data).

Kết quả của mô hình sau khi được train với tham số tương tự:

Max depth	RMSE	R2
3	4924.975609542068	0.007390113874465087
5	4923.82795323045	0.007852671402200806
7	4953.890147573222	-0.004299327847247181

Có thể thấy, cả 2 cách đều cho ra kết quả khá tương đồng với nhau, tuy nhiên, Rdd-Based vẫn cho ra RMSE tốt hơn một chút so với Structured API.

C. Low-Level Operations

(Dữ liệu được đọc từ hdfs)

1. Ý tưởng thực hiện thuật toán hồi quy bằng cây quyết định.

Có nhiều tiêu chí phân tách để chọn đặc trưng ở node. Một số tiêu chí phổ biến và ứng dụng là:

- Entropy (dùng trong ID3, C4.5): Đo lường mức độ "hỗn loạn" trong phân bố lớp, thường dùng cho phân loại.
- Gini Index (dùng trong CART): Đo lường xác suất chọn sai lớp, dùng cho phân loại.
- Variance Reduction (dùng trong CART cho hồi quy): Đo lường mức độ phân tán của nhãn, dùng cho hồi quy.

Ở bài toán này, Variance Reduction sẽ được sử dụng

a. Công thức tính phương sai mẫu

$$\text{variance} = \frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2$$

$$\text{variance} = \frac{1}{N-1} \left(\sum_{i=1}^N x_i^2 - N\mu^2 \right)$$

Phương sai càng nhỏ, các giá trị nhãn càng đồng nhất, nghĩa là nút lá càng "tốt" cho dự đoán.

b. Công thức Variance Reduction

$$\text{Variance Reduction} = \text{Var}(D) - \left(\frac{n_{\text{left}}}{n} \cdot \text{Var}(D_{\text{left}}) + \frac{n_{\text{right}}}{n} \cdot \text{Var}(D_{\text{right}}) \right)$$

Trong đó:

- $\text{Var}(D)$: Phương sai của tập dữ liệu gốc
- $D_{\text{left}}, D_{\text{right}}$: là hai tập con phân tách từ D với
 - D_{left} : các mẫu thỏa $x_j \leq t$
 - D_{right} : các mẫu thỏa $x_j \geq t$
- $\text{Var}(D_{\text{left}}), \text{Var}(D_{\text{right}})$: Phương sai của hai tập con.
- $\frac{n_{\text{left}}}{n}, \frac{n_{\text{right}}}{n}$: Trọng số dựa trên kích thước tập con.

Mục tiêu: tìm đặc trưng j và ngưỡng t sao cho Variance Reduction là lớn nhất

c. Dự đoán tại nút lá

Khi dữ liệu được phân tách thành các nút lá, giá trị dự đoán cho một mẫu rơi vào nút lá là trung bình của các nhãn trong nút lá đó:

$$\hat{y} = \frac{1}{n_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

2. Các bước của thuật toán

Bước 1: Khởi tạo

- Bắt đầu với node gốc chứa toàn bộ dữ liệu D .
- Đặt các tham số:
 - `max_depth`: Độ sâu tối đa của cây (ví dụ: 5).
 - `min_samples_split`: Số mẫu tối thiểu để tiếp tục phân chia (ví dụ: 10).

Bước 2: Kiểm tra điều kiện dừng

Tại mỗi node, kiểm tra:

- Nếu số mẫu $n < \text{min_samples_split}$ hoặc độ sâu đạt max_depth , hoặc không còn đặc trưng nào để chia:
 - Tạo node lá với giá trị dự đoán là trung bình $\bar{y}$.
- Nếu không, tiếp tục tìm điểm phân chia.

Bước 3: Tìm điểm phân chia tối ưu

- Với mỗi đặc trưng j (ví dụ: pickup_longitude , pickup_latitude , ...):
 - Chọn tập hợp các ngưỡng t (thresholds):
 - Lấy 10 ngưỡng dựa trên phân vị (quantiles) để giảm thời gian tính toán (nếu số mẫu bé hơn 10 thì lấy tất cả giá trị làm ngưỡng)
 - Với mỗi ngưỡng t :
 - Chia dữ liệu thành D_{left} và D_{right} .
 - Tính phương sai: $\text{Var}(D_{\text{left}}), \text{Var}(D_{\text{right}})$
 - Tính weighted variance:
$$\text{Weighted Variance} = \frac{n_{\text{left}}}{n} \cdot \text{Var}(D_{\text{left}}) + \frac{n_{\text{right}}}{n} \cdot \text{Var}(D_{\text{right}})$$
 - Tính Variance Reduction:
$$\text{Gain} = \text{Var}(D) - \text{Weighted Variance}$$
 - Chọn j, t có **Gain** lớn nhất ($\text{best_feature}, \text{best_threshold}$).
 - Nếu $\text{Gain} \leq 0$, dừng và tạo node lá.

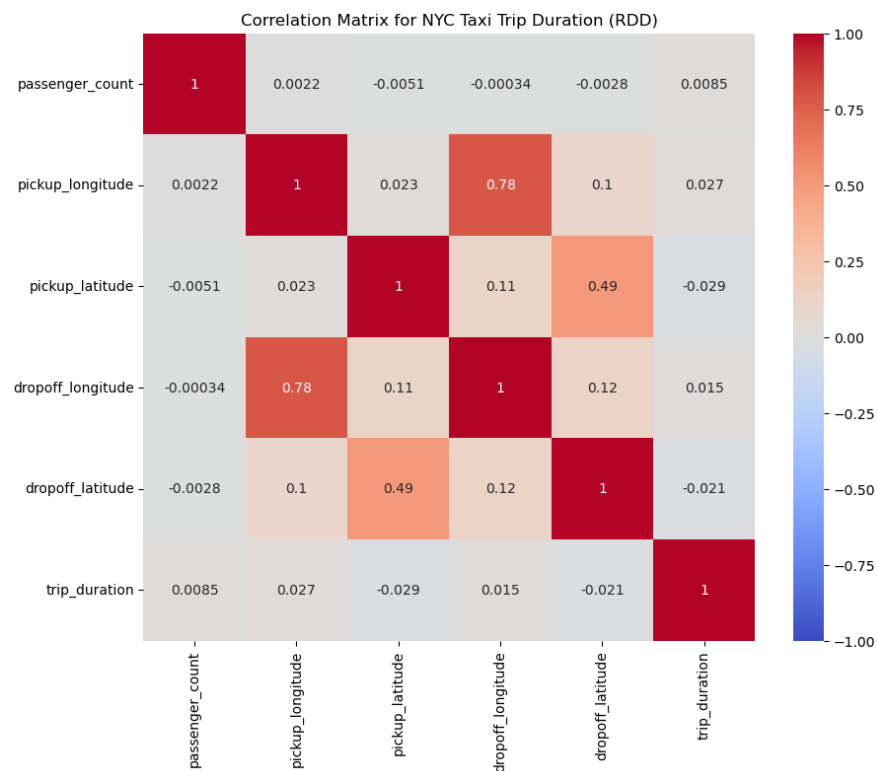
Bước 4: Phân chia đệ quy

- Sử dụng best_feature và best_threshold để chia D thành D_{left} và D_{right} .
- Đệ quy gọi hàm xây dựng cây cho D_{left} và D_{right} , tăng độ sâu lên 1.

Bước 5: Dự đoán

- Với một mẫu mới x :
 - Bắt đầu từ node gốc, duyệt cây theo các điều kiện phân chia ($x_j \leq t$).
 - Khi đến node lá, trả về giá trị $\hat{y}$ (trung bình của node lá).

3. Triển khai code



Từ ma trận tương quan (trong file **CorrelationMatrix.ipynb**), ta chọn các thuộc tính

```
feature_cols = ["pickup_longitude", "pickup_latitude", "dropoff_longitude", "dropoff_latitude"]
train_numeric_indices = [5, 6, 7, 8] # pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude
target_index = 10 # trip_duration
```

Thực hiện tìm hiểu dữ liệu, tính toán một số thống kê cơ bản

Số dòng dữ liệu: 1458644

	Count	Mean	Min	Max	Q1 (25%)	\
pickup_longitude	1458644	-73.973486	-121.933342	-6.133553e+01	-73.991867	
pickup_latitude	1458644	40.750921	34.359695	5.188108e+01	40.737347	
dropoff_longitude	1458644	-73.973416	-121.933304	-6.133553e+01	-73.991325	
dropoff_latitude	1458644	40.751800	32.181141	4.392103e+01	40.735886	
trip_duration	1458644	959.492273	1.000000	3.526282e+06	397.000000	

	Q2 (50%)	Q3 (75%)
pickup_longitude	-73.981743	-73.967331
pickup_latitude	40.754101	40.768360
dropoff_longitude	-73.979752	-73.963013
dropoff_latitude	40.754524	40.769810
trip_duration	662.000000	1075.000000

Số dòng dữ liệu bị thiếu: 0

Số dòng trùng lặp: 9

[Stage 41:>

(0 + 2) / 2]

Số dòng sau khi xóa trùng: 1458635

Phân chia dữ liệu thành tập train (80%) và tập validation (20%)

```
# Phân chia tập train và validation
train_split, val_split = rdd_distinct.randomSplit([0.8, 0.2], seed=22120071)
```

Lớp và các hàm chính

1. Hàm compute_variance: tính phương sai và số lượng

```
def compute_variance(data_rdd, isSample=True):
    """
    Tính phương sai của nhân (target) trong RDD
    Args:
        data_rdd: RDD chứa dữ liệu [[features], target]
        isSample: True nếu là mẫu (sample), False nếu là tổng thể (population)
    Returns:
        tuple: (phương sai, số lượng mẫu) # trả về số lượng mẫu để bên ngoài sử dụng hàm không mất thời gian tính lại
    """
    count = data_rdd.count()

    if count == 0:
        return 0.0, 0

    sum_y, sum_y_squared = data_rdd.map(lambda x: (x[1], x[1] ** 2)).reduce(lambda a, b: (a[0] + b[0], a[1] + b[1]))
    mean_y = sum_y / count

    variance = 0
    if isSample:
        variance = (sum_y_squared - count * mean_y ** 2) / (count - 1)
    else:
        variance = (sum_y_squared - count * mean_y ** 2) / count
    return variance, count
```

2. Hàm find_best_split: Tìm cách chia tốt nhất cho một node trong cây quyết định

```
def find_best_split(data_rdd, feature_indices):  
    """  
    Tìm cách chia tốt nhất cho một node trong cây quyết định  
  
    Args:  
        data_rdd: RDD chứa dữ liệu  
        feature_indices: Danh sách chỉ số các feature cần xem xét  
  
    Returns:  
        tuple: (best_feature, best_threshold, best_gain (AKA variance reduction))  
    """
```

3. Class Node

```
class Node:  
    def __init__(self, feature_idx=None, threshold=None, left=None, right=None, value=None):  
        self.feature_idx = feature_idx  
        self.threshold = threshold  
        self.left = left  
        self.right = right  
        self.value = value # Giá trị dự đoán (mean của nhãn)
```

4. Hàm build_tree: xây dựng cây quyết định

```
def build_tree(data_rdd, feature_indices, max_depth, min_samples_split=10, depth=0):  
    """  
    Xây dựng cây quyết định đệ quy  
  
    Args:  
        data_rdd: Dữ liệu huấn luyện  
        feature_indices: Danh sách các feature  
        max_depth: Độ sâu tối đa của cây  
        min_samples_split: Số mẫu tối thiểu để tiếp tục chia  
        depth: Độ sâu hiện tại  
  
    Returns:  
        Node: Node gốc của cây  
    """
```

5. Hàm predict_tree: hàm dự đoán kết quả sau khi huấn luyện cây xong

```
# Hàm dự đoán cho một điểm dữ liệu  
def predict_tree(node, features):  
    """  
    Dự đoán giá trị cho một điểm dữ liệu bằng cách duyệt cây quyết định  
  
    Args:  
        node: Node hiện tại đang xét (thường là root)  
        features: giá trị các đặc trưng của điểm dữ liệu cần dự đoán  
  
    Returns:  
        float: Giá trị dự đoán  
    """
```

6. Hàm compute_metrics: tính toán các chỉ số đánh giá mô hình

```
# Đánh giá mô hình
def compute_metrics(predictions_rdd):
    """
    Tính các chỉ số đánh giá hiệu suất mô hình

    Args:
        predictions_rdd: RDD chứa các cặp (y_true, y_pred)

    Returns:
        tuple: (RMSE, MAE, R²)
    """
```

Dự đoán trên tập validation và in ra một vài kết quả dự đoán

Dự đoán trên tập validation

```
# val_split có dạng [(features, target), ...]
# val_predictions có dạng [(target, predicted_value), ...]
val_predictions = val_split.map(lambda x: (x[1], predict_tree(tree, x[0])))
```

In ra một vài mẫu so sánh giữa kết quả dự đoán và kết quả thực tế trên tập Validation

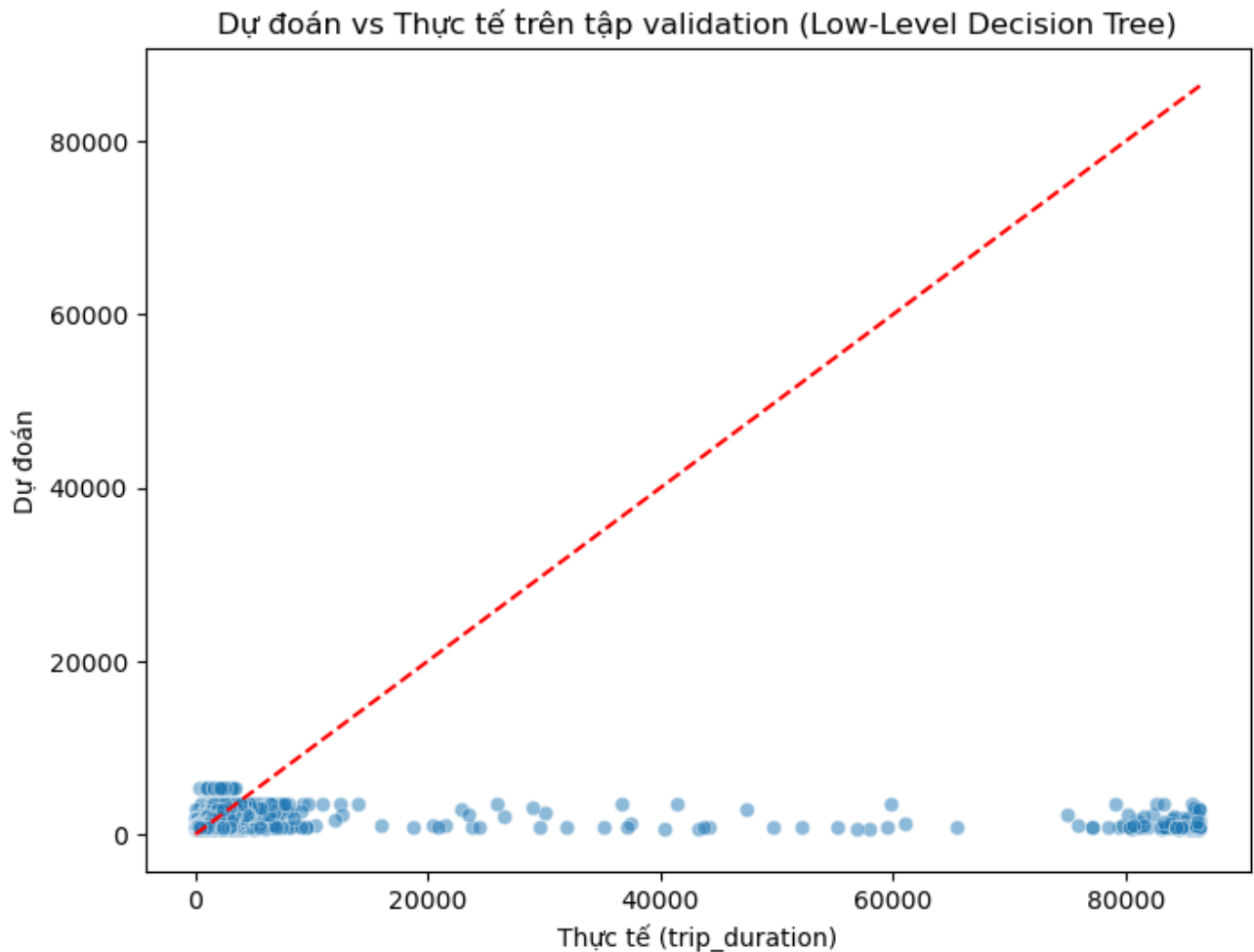
```
for target, predicted_value in val_predictions.take(5):
    print(f"Target: {target}, Predicted: {predicted_value}")
```

8]

```
Target: 1414.0, Predicted: 753.3961339466949
Target: 707.0, Predicted: 788.3887543082226
Target: 526.0, Predicted: 1137.6423483455883
Target: 721.0, Predicted: 753.3961339466949
Target: 1003.0, Predicted: 1562.9956129561297
25/05/09 23:27:01 WARN BlockManager: Task 11706 already completed, not releasing lock for rdd_60_0
```

Đánh giá dự trên tập validation

```
≡ Đánh giá mô hình trên tập validation ≡
RMSE: 2975.75
MAE: 488.79
R²: 0.02
```



Trực quan hình ảnh cây quyết định: *Xem tại file CorrelationMatrix.ipynb*

Chạy dự đoán trên tập test

```
# Tạo riêng chỉ số cột features cho tập test vì khác với tập train
test_numeric_indices = [4, 5, 6, 7] # pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude
def parse_test_line_with_id(line):
    try:
        values = line.split(",")
        id_val = values[0] # Cột id
        features = [float(values[i]) for i in test_numeric_indices]
        if any(v is None for v in features):
            return None
        return (id_val, features)
    except:
        return None

# Parse test data để giữ id
test_rdd = test_data.map(parse_test_line_with_id).filter(lambda x: x is not None).cache()

# Dự đoán
predictions = test_rdd.map(lambda x: (x[0], predict_tree(tree, x[1])))
```

In ra một vài mẫu dự đoán

```
for id, value in predictions.take(10):  
    print(f"ID: {id}, Dự đoán: {value:.2f}")
```

```
ID: id3004672, Dự đoán: 753.40  
ID: id3505355, Dự đoán: 1137.64  
ID: id1217141, Dự đoán: 974.28  
ID: id2150126, Dự đoán: 753.40  
ID: id1598245, Dự đoán: 753.40  
ID: id0668992, Dự đoán: 753.40  
ID: id1765014, Dự đoán: 1337.79  
ID: id0898117, Dự đoán: 788.39  
ID: id3905224, Dự đoán: 1852.10  
ID: id1543102, Dự đoán: 753.40
```

Thách thức khi mở rộng phương pháp Decision Tree với Variance Reduction cho cây sâu hơn và rừng cây (ensemble)

1. Cây sâu hơn:

- **Tốn tài nguyên tính toán:** Cây sâu hơn (tăng *max_depth*) làm tăng số node cần xử lý theo cấp số nhân ($2^{\text{max_depth}}$). Mỗi node yêu cầu tính phương sai và thử nhiều ngưỡng, gây áp lực lớn lên CPU và bộ nhớ, đặc biệt với dữ liệu lớn (~1.4M dòng).
- **Overfitting:** Cây sâu dễ học nhiều, đặc biệt khi chỉ dùng 4 đặc trưng như trong code. Điều này làm giảm khả năng tổng quát hóa trên tập test.
- **Tốn thời gian trên Spark:** Với RDD, các thao tác *filter*, *reduce*, *count* lặp lại nhiều lần cho mỗi node, dẫn đến thời gian xử lý tăng đáng kể.

2. Rừng cây (Random Forest):

- **Tăng chi phí tính toán:** Rừng cây yêu cầu huấn luyện nhiều cây. Mỗi cây cần một mẫu bootstrap và tập đặc trưng ngẫu nhiên, làm tăng gấp bội lượng tính toán. Trên Spark, điều này đòi hỏi quản lý nhiều RDD song song, dễ gây tắc nghẽn.
- **Quản lý bộ nhớ:** Nhiều cây cần lưu trữ đồng thời trong bộ nhớ và RAM (yêu cầu bộ nhớ và RAM lớn). Spark có thể dùng swap hoặc spill sang disk, làm chậm hiệu suất.
- **Độ phức tạp song song hóa:** Random Forest cần phân phối huấn luyện cây trên các executor. Nếu số phân vùng hoặc executor không tối ưu, hiệu quả song song giảm, dẫn đến thời gian chạy lâu.
- **Kết hợp dự đoán:** Tổng hợp dự đoán từ nhiều cây (ví dụ: trung bình) đòi hỏi thêm bước *reduce*, tăng chi phí trên dữ liệu lớn.

3. Giải pháp:

- **Cắt tỉa cây:** Giới hạn *max_depth* hoặc dùng *min_samples_split* lớn để giảm độ sâu.
- **Tối ưu Spark:** Tăng cache, giảm số ngưỡng, và điều chỉnh số phân vùng phù hợp.
- **Rừng cây:** Dùng mẫu nhỏ hơn cho mỗi cây, chọn số cây vừa đủ, và tận dụng API MLlib của Spark để tối ưu hóa Random Forest.

So sánh hiệu suất của Decision Tree với các phương pháp

Tiêu chí	Low-level RDD-Based	MLlib RDD-Based	Structured API
Hiệu suất	Với <i>max_depth</i> = 5 RMSE: 2975.75 MAE: 488.79 R ² : 0.02	Với <i>max_depth</i> = 3 RMSE: 4924.975609 R ² : 0.007390 Với <i>max_depth</i> = 5 RMSE: 4923.827953 R ² : 0.007853 Với <i>max_depth</i> = 7 RMSE: 4953.890147 R ² : -0.004299	Với <i>max_depth</i> = 3 RMSE: 6288.53 R ² : 0.004556 Với <i>max_depth</i> = 5 RMSE: 6288.359505 R ² : 0.00461079 Với <i>max_depth</i> = 7 RMSE: 6333.888542 R ² : -0.009855
Thời gian huấn luyện	Khoảng 30 phút	Từ 3 đến 5 phút	Dưới 1 phút
Khả năng mở rộng	Kém: Tính toán thủ công, khó mở rộng với cây sâu hoặc rừng cây, dễ lỗi bộ nhớ trên dữ liệu lớn.	Trung bình: Tốt hơn low-level, nhưng kém Structured API do xử lý RDD thủ công.	Tốt: Hỗ trợ pipeline, Random Forest, và phân phối trên cluster (YARN), tối ưu với Catalyst.
Dễ sử dụng & bảo trì	Kém: Code phức tạp, dễ lỗi, khó thêm đặc trưng hoặc bảo trì.	Trung bình: Dễ hơn low-level, nhưng cần xử lý RDD, ít linh hoạt hơn DataFrame.	Tốt: API cấp cao, dễ tích hợp pipeline, tuning tham số, và bảo trì.